

JAVA FULL STACK INTERVIEW BIBLE

Personalized Preparation Handbook

Prepared for: **VIRAJ PAWAR**

Java Full Stack Developer | Spring Boot | React.js | Microservices

3.5 Years Experience | HSBC Banking | Capgemini

Oracle Certified Java SE 21 | AWS Cloud Practitioner

Target Companies: TCS | Infosys | Accenture | Capgemini | Cognizant
Deloitte | IBM | Siemens | HSBC | JPMorgan | Oracle

TABLE OF CONTENTS

- Section 1 — Resume Analysis & ATS Score
- Section 2 — Skill Gap Analysis
- Section 3 — Complete Java Roadmap
- Section 4 — Spring & Spring Boot Master Guide
- Section 5 — React.js Complete Interview Guide
- Section 6 — Database & SQL Master Section
- Section 7 — Microservices & System Design
- Section 8 — Project Deep Dive
- Section 9 — Must-Know Interview Questions (700+ Q&A;)
- Section 10 — Scenario-Based Questions
- Section 11 — Coding Round Preparation
- Section 12 — HR & Managerial Round Preparation
- Section 13 — Daily Study Plan (30/60-Day Roadmap)
- Section 14 — Final Rapid Revision Notes & Cheat Sheets

SECTION 1 | RESUME ANALYSIS & ATS SCORE

Overall Impression

Viraj's resume is **well-structured, ATS-friendly, and clearly targeted** at Java Full Stack Developer roles. With 3.5 years at Capgemini on the HSBC banking platform, two personal projects, Oracle Java SE 21 certification, and AWS Cloud Practitioner, it signals both domain depth and a commitment to self-improvement. The resume reads as a genuine **mid-level to senior engineer profile**.

ATS SCORE ESTIMATION	82 / 100
Keyword Density	Strong (Java, Spring Boot, React, Microservices, JWT, Redis, Docker, K8s, JUnit, CI/CD)
Quantifiable Achievements	Moderate (12+ microservices, 85% coverage — needs more metrics)
Action Verbs	Good (Contributed, Developed, Implemented, Introduced, Fixed, Wrote)
Section Clarity	Excellent (Summary, Experience, Projects, Skills, Certs, Education)
Role Alignment	High — Java Full Stack / Backend Eng / Spring Boot Developer
Length	Ideal (2 pages)
Missing Keywords	GraphQL, Kafka Producer/Consumer, TestContainers, ELK Stack, CI/CD depth

Resume Strengths

- Real banking domain (HSBC) — highly attractive to BFSI-facing companies.
- Clear tech stack: Java 17/21, Spring Boot, React.js, MySQL, Redis, Docker, Kubernetes.
- Security expertise: JWT, Spring Security, RBAC — critical for enterprise hiring.
- DevOps touch: Docker, Kubernetes (Helm, rolling deployments), Jenkins CI pipelines.
- Certifications: OCP Java SE 21 + AWS CCP — differentiators at screening stage.
- Testing discipline: JUnit 5, Mockito, 85% code coverage — valued by senior engineers.
- Personal projects show initiative (Job Portal with full-stack + Docker).
- High CGPA (9.11/10) from a recognized technical university.

Resume Weaknesses & Improvements

Weakness	Recommended Improvement
Weak metrics	Add throughput numbers: 'Reduced API response time by 35%', 'Handled 10K TPS'.
No Kafka depth	Mentioned as skill but not in experience. Add even basic usage or remove.

No GraphQL	Many product companies ask for it; add to your learning backlog.
No cloud deployment specifics	Mention AWS EKS/ECR or any cloud-specific service you used at HSBC.
Soft skills section missing	Consider adding a line about mentoring junior devs if applicable.
Cab project is legacy	Java Servlets/JSP signals older tech — frame it as foundational; lead with Job Portal.

Best-Fit Target Companies

Company	Why You Fit	Likely Difficulty
HSBC / Barclays / Standard Chartered	Direct banking domain match	Medium
Capgemini (other clients)	Already internal — strong referral path	Low-Medium
JPMorgan / Goldman Sachs Tech	Spring Boot + Microservices + Security	High
TCS / Infosys / Wipro	Volume hiring, your stack is perfect	Low-Medium
Cognizant / Accenture	Banking + Full-stack combo very sought	Medium
Oracle India	Java OCP cert is a differentiator	Medium-High
IBM / Deloitte Tech	Enterprise microservices exp. valued	Medium
Siemens / Bosch (IT)	Embedded enterprise apps fit your arc	Medium

SECTION 2 | SKILL GAP ANALYSIS

The table below rates each skill on a 1–5 scale and maps the gap to priority actions.

Skill	Your Level	Market Expected	Gap	Priority	Action Plan
Core Java (8–21)	4/5	4/5	Low	MEDIUM	Revise Records, Sealed Classes, Pattern Matching
Spring Boot	4/5	5/5	Low	MEDIUM	Deep dive: Auto-Config, Custom Starters, Actuator
Spring Security / JWT	4/5	4/5	Low	LOW	Study OAuth2 flows in depth
Microservices	3/5	4/5	Medium	HIGH	Practice Saga, CQRS, service mesh concepts
React.js	3/5	4/5	Medium	HIGH	Learn Redux Toolkit, React Query, Testing Library
JPA / Hibernate	3/5	4/5	Medium	HIGH	Master fetch strategies, L2 cache, criteria API
Kafka	2/5	4/5	High	HIGH	Build producer/consumer; study partitions, offsets
System Design	2/5	4/5	High	HIGH	Study HLD/LLD, design patterns for distributed systems
DSA / Coding Rounds	2/5	4/5	High	HIGH	30-min daily LeetCode: arrays, trees, DP basics
Docker / K8s	3/5	3/5	Low	LOW	Already good; study K8s RBAC, HPA, resource limits
SQL / Query Opt.	3/5	4/5	Medium	MEDIUM	Practice window functions, execution plans, indexes
Redis	3/5	3/5	Low	LOW	Study eviction policies, pub/sub, data structures
AWS	2/5	3/5	Medium	MEDIUM	Study EC2, S3, RDS, ECS/EKS, IAM beyond CCP
GraphQL	1/5	2/5	Low-Med	LOW	Learn schema, resolvers, mutations — quick win
Elasticsearch / ELK	1/5	2/5	Low-Med	LOW	Understand indexing basics; used with Splunk exposure
TestContainers	1/5	2/5	Medium	MEDIUM	Integration test DB/Kafka in real containers

TOP 3 IMMEDIATE PRIORITIES: (1) Kafka deep dive — appears in 80% of senior Java interviews, (2) System Design fundamentals — asked at all levels 3Y+, (3) DSA daily practice — filter round at product companies.

SECTION 3 | COMPLETE JAVA ROADMAP

Core Java & OOPs

Encapsulation, Inheritance, Polymorphism (compile-time via overloading, runtime via overriding), Abstraction. Abstract class vs Interface: abstract classes allow state and constructors; interfaces define contracts. Java 8+ interfaces support default and static methods.

Q: What is the difference between abstract class and interface?

A: Abstract class can have constructors, instance variables, and method implementations. Interfaces define a contract; since Java 8 they support default/static methods; since Java 9, private methods. Use abstract class when sharing code between closely related classes; interface for unrelated classes sharing behavior.

Q: What is method overriding vs overloading?

A: Overloading: same method name, different parameter list — resolved at compile time (static polymorphism). Overriding: subclass redefines superclass method with same signature — resolved at runtime (dynamic polymorphism).

Q: Explain SOLID principles with real examples.

A: S-Single Responsibility: UserService only handles user logic, not emails. O-Open/Closed: Add new payment types via strategy pattern without changing PaymentProcessor. L-Liskov Substitution: Any subtype can replace its parent without breaking code. I-Interface Segregation: Don't force classes to implement methods they don't need. D-Dependency Inversion: Depend on abstractions (interfaces), not concrete implementations.

Code Example:

```
class ImmutableUser { private final String name; private final int age; public
ImmutableUser(String n, int a){this.name=n; this.age=a;} public String getName(
){return name;} }
```

Collections Framework

ArrayList (dynamic array, $O(1)$ get), LinkedList (doubly-linked, $O(1)$ add/remove at ends), HashMap (hash table, $O(1)$ avg), TreeMap (Red-Black tree, $O(\log n)$, sorted), HashSet (backed by HashMap), LinkedHashMap (insertion order). ConcurrentHashMap uses segment/bucket locking for thread safety without blocking entire map.

Q: HashMap vs ConcurrentHashMap — when to use which?

A: HashMap is not thread-safe; use in single-threaded contexts. ConcurrentHashMap allows concurrent reads without locking and uses fine-grained locking for writes — ideal for multi-threaded environments like Spring beans with shared state.

Q: How does HashMap handle collisions in Java 8+?

A: Java 8 introduced treeification: when a bucket exceeds 8 entries and the table has 64+ buckets, the linked list is converted to a Red-Black tree, reducing worst-case lookup from $O(n)$ to $O(\log n)$.

Q: What is the difference between fail-fast and fail-safe iterators?

A: Fail-fast (ArrayList, HashMap): throws ConcurrentModificationException if collection is modified during iteration. Fail-safe (CopyOnWriteArrayList, ConcurrentHashMap): iterates over a snapshot; no exception but may not reflect latest data.

Code Example:

```
Map freq = new HashMap<>();
words.forEach(w -> freq.merge(w, 1, Integer::sum));
```

Java 8+ Features — Streams, Lambdas, Optionals

Lambda expressions provide concise anonymous function syntax. Stream API enables declarative data processing with lazy evaluation. Optional avoids NullPointerException. CompletableFuture enables async programming. Java 14+ Records provide immutable data classes. Java 17 Sealed Classes restrict inheritance hierarchies.

Q: What is the difference between map() and flatMap() in streams?

A: map() applies a 1-to-1 transformation: each element produces one output. flatMap() applies a 1-to-many transformation and flattens the result: e.g., List<List> flattened to List.

Q: Explain Functional Interfaces. Name four built-in ones.

A: A functional interface has exactly one abstract method. Built-in examples: Predicate (T->boolean), Function (T->R), Consumer (T->void), Supplier (()->T). Annotated with @FunctionalInterface for compile-time verification.

Q: What is method reference? Give an example from a real project.

A: Method reference (::) is shorthand for a lambda that calls an existing method. In banking, userList.stream().map(User::getEmail).collect(Collectors.toList()) replaces .map(u -> u.getEmail()).

Code Example:

```
List emails = users.stream()
    .filter(u -> u.isActive())
    .map(User::getEmail)
    .distinct()
    .collect(Collectors.toList());
```

Multithreading & Concurrency

Thread lifecycle: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. synchronized keyword provides mutual exclusion. volatile ensures visibility across threads. Executor framework (ThreadPoolExecutor) manages thread pools. CompletableFuture chains async tasks. ReentrantLock provides explicit locking with fairness options.

Q: What is the difference between synchronized and ReentrantLock?

A: synchronized is simpler but inflexible — no timeout, no fairness policy, no interruptible locking. ReentrantLock supports tryLock() with timeout, lockInterruptibly(), and optional fair ordering. In HSBC's transaction processing, ReentrantLock with tryLock() prevents deadlocks when acquiring multiple locks.

Q: Explain volatile vs synchronized.

A: volatile guarantees visibility: all reads/writes go directly to main memory, preventing CPU caching. synchronized guarantees both visibility AND atomicity for compound operations. Use volatile for simple flags (isRunning = false); use synchronized for read-modify-write.

Q: What is a deadlock? How do you prevent it?

A: Deadlock: Thread A holds Lock 1, waits for Lock 2; Thread B holds Lock 2, waits for Lock 1 — circular wait. Prevention: consistent lock ordering, use tryLock() with timeout, avoid nested locks, use higher-level abstractions like ConcurrentHashMap.

Code Example:

```
CompletableFuture.supplyAsync(() -> fetchAccount(id))
    .thenApply(acc -> calculateBalance(acc))
    .thenAccept(bal -> notify(bal))
    .exceptionally(ex -> { log(ex); return null; });
```

JVM Internals & Memory Management

JVM memory: Heap (Young Gen: Eden + Survivor; Old Gen), Stack (per thread, local vars, method frames), Metaspace (class metadata, replaces PermGen since Java 8), Code Cache. Garbage Collection: G1GC (default Java 9+) divides heap into regions, good for low latency. ZGC (Java 15+) aims for sub-millisecond pauses. String Pool stores interned strings in Metaspace.

Q: What is the difference between stack and heap memory?

A: Stack: thread-local, stores primitives and references, LIFO, automatically cleaned when method returns. Heap: shared across threads, stores objects, managed by GC. A NullPointerException comes from a null reference on the stack pointing to a non-existent heap object.

Q: How do you diagnose a memory leak in production?

A: At HSBC: (1) Monitor heap via Splunk JVM metrics or JConsole, (2) capture heap dump with jmap -dump:format=b,file=heap.hprof, (3) analyze in Eclipse MAT — look for retained heap, GC roots holding references unexpectedly (common: static collections, ThreadLocal not removed).

Q: Explain G1GC and when you'd tune it.

A: G1GC divides heap into equal-size regions (1-32MB), prioritizes garbage collection in regions with most garbage. Key tuning: -XX:MaxGCPauseMillis=200 (target pause), -XX:G1HeapRegionSize, -Xms/-Xmx. In banking apps with strict SLAs, tuning pause targets is critical.

Code Example:

```
-Xms512m -Xmx2g -XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/logs/heap.hprof
```

Design Patterns

Creational: Singleton (one instance), Factory (delegate creation), Builder (complex object construction). Structural: Adapter (interface compatibility), Decorator (add behavior), Proxy (access control). Behavioral: Strategy (interchangeable algorithms), Observer (event notification), Template Method. In Spring: Singleton beans, Factory via ApplicationContext, Proxy via AOP, Template via JdbcTemplate.

Q: How is Singleton implemented in a thread-safe way in Java?

A: Double-checked locking with volatile: if(instance==null){ synchronized(Cls.class){ if(instance==null) instance=new Cls(); } } Or better: Bill Pugh's Holder pattern using inner static

class — lazy, thread-safe, no synchronization overhead.

Q: When would you use Strategy pattern in a banking application?

A: Payment processing: PaymentStrategy interface with CreditCardPayment, UPIPayment, NetBankingPayment implementations. The checkout service holds a reference to PaymentStrategy — switch payment type at runtime without changing service logic.

Q: Explain Builder pattern with a real example.

A: When constructing a LoanApplication object with 15+ optional fields, Builder pattern avoids telescoping constructors. Lombok's @Builder annotation generates it automatically in Spring Boot services.

Code Example:

```
// Bill Pugh Singleton
public class Config {
    private Config(){}
    private static class Holder { static final Config INSTANCE = new Config(); }
    public static Config get(){ return Holder.INSTANCE; }
}
```

Exception Handling

Checked exceptions (IOException, SQLException) must be handled or declared. Unchecked (RuntimeException subclasses) don't require explicit handling. Best practices: never swallow exceptions, log with full stack trace, use custom exception hierarchy, leverage try-with-resources for AutoCloseable.

Q: What is the difference between checked and unchecked exceptions?

A: Checked: compiler enforces handling — use for recoverable conditions (file not found, DB connection failed). Unchecked: RuntimeException subclasses — use for programming errors (null pointer, illegal argument). In Spring Boot, most service exceptions should be unchecked; translate to HTTP responses via @ControllerAdvice.

Q: How do you implement global exception handling in Spring Boot?

A: @ControllerAdvice + @ExceptionHandler: centralized handler returns consistent error DTO. At HSBC, we used a ErrorResponse POJO with timestamp, status, message, path, and ProblemDetail (RFC 7807 support in Spring 6) for client-parseable error contracts.

Code Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity handle(ResourceNotFoundException ex){
        return ResponseEntity.status(404).body(new ErrorResponse(ex.getMessage()));
    }
}
```

SECTION 4 | SPRING & SPRING BOOT MASTER GUIDE

Spring Core — IoC & Dependency Injection

Inversion of Control: the framework manages object creation and lifecycle instead of the application. DI types: Constructor (recommended — immutable, testable), Setter (optional deps), Field (@Autowired — avoid in production code). Bean scopes: Singleton (default), Prototype, Request, Session, Application.

Q: What is the difference between @Component, @Service, @Repository, @Controller?

A: All are specializations of @Component — Spring treats them identically for component scanning. Semantic difference: @Repository enables exception translation (DataAccessException); @Controller marks MVC controllers; @Service marks business logic — aids readability and future tooling.

Q: Explain Bean lifecycle in Spring.

A: 1) Instantiation, 2) Populate properties/DI, 3) BeanNameAware.setBeanName(), 4) BeanPostProcessor.postProcessBeforeInitialization(), 5) @PostConstruct / InitializingBean.afterPropertiesSet(), 6) BeanPostProcessor.postProcessAfterInitialization(), 7) Bean in use, 8) @PreDestroy / DisposableBean.destroy() on context close.

Code Example:

```
@Service
@RequiredArgsConstructor
public class AccountService {
    private final AccountRepository repo; // Constructor DI via Lombok
    public Account findById(Long id){
        return repo.findById(id).orElseThrow(() -> new AccountNotFoundException(id)
    );
    }
}
```

Spring Boot Architecture & Auto-Configuration

@SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan. Auto-configuration reads spring.factories / AutoConfiguration.imports, applies conditional beans (@ConditionalOnClass, @ConditionalOnProperty). Embedded server (Tomcat/Undertow/Jetty) eliminates WAR deployment. application.yml / application.properties + @ConfigurationProperties bind config.

Q: How does Spring Boot Auto-Configuration work?

A: Spring Boot scans META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports. Each auto-config class uses conditions: if DataSource class is on classpath and no DataSource bean exists, DataSourceAutoConfiguration creates one from spring.datasource.* properties.

Q: How do you externalize configuration in Spring Boot for different environments?

A: Use application-{profile}.yaml (dev, staging, prod). Activate via spring.profiles.active=prod or SPRING_PROFILES_ACTIVE env var. For sensitive data (DB passwords, JWT secrets), use environment variables or HashiCorp Vault integration via spring-cloud-vault.

Code Example:

```
# application-prod.yml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate.ddl-auto: validate
```

Spring Security — JWT & RBAC

Spring Security filter chain intercepts every request. JWT flow: client sends credentials -> server validates, returns signed JWT -> client includes JWT in Authorization: Bearer header -> server validates signature and claims. RBAC: @PreAuthorize("hasRole(ADMIN)") on service methods. At HSBC: separate service accounts per microservice, short-lived tokens (15 min), refresh token rotation.

Q: How do you secure a REST API with JWT in Spring Boot?

A: 1) Add JwtAuthenticationFilter extending OncePerRequestFilter — extract and validate token. 2) On valid token, set SecurityContextHolder with UsernamePasswordAuthenticationToken. 3) Configure SecurityFilterChain: permit /auth/**, require authentication for others. 4) Generate token with JJWT/Nimbus: HS256 or RS256, include userId, roles, expiry.

Q: What is the difference between Authentication and Authorization?

A: Authentication: who are you? (verify credentials, issue token). Authorization: what can you do? (check permissions for the requested resource). In Spring Security: AuthenticationManager handles authentication; AccessDecisionManager / method security handles authorization.

Code Example:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http.csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers('/api/auth/**').permitAll()
            .anyRequest().authenticated())
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Microservices with Spring Cloud

Spring Cloud components: Eureka (service discovery — services register, clients discover), Spring Cloud Gateway (API gateway — routing, rate limiting, auth filter), OpenFeign (declarative HTTP client between microservices), Resilience4j (Circuit Breaker — CLOSED, OPEN, HALF_OPEN states), Spring Cloud Config (centralized config from Git).

Q: Explain Circuit Breaker pattern. When does it OPEN?

A: Circuit Breaker monitors downstream call failure rate. CLOSED: normal operation. OPEN: after failure threshold (e.g., 50% in 5 calls) — subsequent calls fail fast, fallback invoked. HALF_OPEN: after wait duration, allows limited calls to test recovery. At HSBC, transaction service had CB around fraud-check service — fallback allowed transaction with audit flag.

Q: Feign Client vs RestTemplate vs WebClient — when to use which?

A: Feign: declarative, ideal for synchronous service-to-service calls in microservices.
RestTemplate: blocking, being deprecated — use for legacy Spring MVC apps. WebClient: reactive, non-blocking — use with Spring WebFlux or when high concurrency/streaming needed.

Code Example:

```
@FeignClient(name='fraud-service', fallbackFactory=FraudFallback.class)
public interface FraudClient {
    @GetMapping('/api/check/{txnId}')
    FraudResult check(@PathVariable String txnId);
}
```

JPA & Hibernate Deep Dive

JPA entity states: Transient, Persistent (managed by context), Detached, Removed. Fetch strategies: LAZY (proxy, loaded on access), EAGER (loaded with entity). N+1 problem: fetching 100 orders each loading customer separately = 101 queries. Fix: JOIN FETCH in JPQL, @EntityGraph, or batch fetching (@BatchSize). L1 cache: per EntityManager (session). L2 cache: shared across sessions (e.g., Ehcache, Caffeine).

Q: How did you solve the N+1 problem at HSBC?

A: Identified via Hibernate show_sql=true logs — saw 1 account query + 100 transaction queries. Fixed with @EntityGraph(attributePaths={"transactions"}) on repository method, reducing to a single JOIN query. Also configured spring.jpa.properties.hibernate.default_batch_fetch_size=25 for collections that couldn't use join fetch.

Q: What is the difference between @OneToMany(mappedBy) and the owning side?

A: The owning side has the foreign key in the DB and controls persistence. mappedBy on the inverse side means 'the other entity owns this relationship'. Mistakes: setting relationship only on mappedBy side won't persist — always set the owning side.

Code Example:

```
@Query('SELECT a FROM Account a JOIN FETCH a.transactions WHERE a.customerId = :id')
List findWithTxns(@Param('id') Long id);
```

Redis Caching

Redis: in-memory, key-value store. Spring Cache abstraction: @Cacheable, @CacheEvict, @CachePut. TTL prevents stale data. Serialization: use JSON for cross-language compatibility. At HSBC: Redis caching account summaries reduced DB load by 60% on read-heavy dashboards. Cache aside pattern: application checks Redis first; on miss, loads from DB and caches.

Q: What cache eviction strategies does Redis support?

A: noeviction (error when full), allkeys-lru (evict least recently used), volatile-lru (evict LRU among keys with TTL), allkeys-lfu (least frequently used), allkeys-random. For banking read-heavy data:

allkeys-lru with appropriate TTL (300s for account summary).

Q: How do you handle cache invalidation?

A: @CacheEvict on write operations ensures cache consistency. For distributed microservices, use Redis pub/sub or Kafka events to notify other service instances to invalidate. At HSBC, account balance update published to Kafka topic; cache eviction consumer invalidated Redis entries across pods.

Code Example:

```
@Cacheable(value='accountSummary', key='#accountId', unless='#result == null')
public AccountDTO getAccountSummary(Long accountId) {
    return accountRepo.findSummaryById(accountId);
}
```

Kafka Basics (Key Gap Area)

Kafka: distributed event streaming. Topics partitioned for parallelism. Producer publishes to topic. Consumer reads from offset (committed position). Consumer groups: each partition assigned to one consumer in group — enables horizontal scaling. At-least-once vs exactly-once delivery. Kafka Connect for data pipelines. In banking: transaction events, audit logs, notification triggers.

Q: What is the difference between at-least-once and exactly-once in Kafka?

A: At-least-once: producer retries on failure — duplicates possible. Consumer must be idempotent. Exactly-once: use Kafka transactions (enable.idempotence=true, transactional.id). For financial transactions, implement idempotency key on consumer side to detect duplicates.

Q: What happens when a consumer crashes before committing an offset?

A: The broker retains the message at uncommitted offset. When consumer restarts or another consumer in group takes over, it re-reads from the last committed offset — messages processed again (at-least-once). Fix: smaller batch size, shorter commit interval, idempotent consumer logic.

Code Example:

```
@KafkaListener(topics='txn-events', groupId='notification-svc')
public void consume(TransactionEvent event) {
    if(!processed.contains(event.getId())) {
        notify(event); processed.add(event.getId());
    }
}
```

SECTION 5 | REACT.JS COMPLETE INTERVIEW GUIDE

Q: What is the Virtual DOM and how does React use it?

A: React maintains a lightweight in-memory representation (Virtual DOM). On state change, React creates a new VDOM, diffs it against the previous one (reconciliation using fiber algorithm), and applies only the minimal real DOM updates. This batching makes UI updates efficient — critical in data-heavy banking dashboards.

Q: Explain `useState` vs `useReducer` — when to use which?

A: `useState`: simple, independent state values. `useReducer`: complex state with multiple sub-values or when next state depends on previous — similar to Redux locally. Example: multi-step loan application form uses `useReducer` with `step`, `formData`, `errors` in one state object.

Q: What is the purpose of `useEffect` and its dependency array?

A: `useEffect` runs side effects after render. Empty `[]`: run once on mount (fetch initial data). With `deps`: run when any dep changes. No `deps`: run after every render (usually unwanted). Cleanup function returned from `useEffect` prevents memory leaks: cancel API calls, clear timers.

Q: Explain `useMemo` and `useCallback` — what problem do they solve?

A: `useMemo`: memoizes a computed value — recalculate only when dependencies change. `useCallback`: memoizes a function reference — prevents child re-renders when passing callbacks as props. Over-using these is a code smell; profile first with React DevTools.

Q: How does Context API differ from Redux?

A: Context API: built-in, good for infrequently-changing global state (theme, auth user). Re-renders all consumers on every context change — use `React.memo` to optimize. Redux: for complex, frequently-updating state with predictable update patterns (shopping cart, transaction filters). Redux Toolkit reduces boilerplate significantly.

Q: How do you handle authentication flow in React?

A: On login: `POST /auth/login`, receive JWT, store in memory (or `httpOnly` cookie — most secure). Axios interceptor attaches Bearer token to every request. Context/Redux stores auth state. Protected routes: checks auth state; redirects to `/login` if unauthenticated. On 401 response, interceptor triggers token refresh or logout.

Q: What is code splitting and lazy loading in React?

A: `React.lazy()` + `Suspense` enables component-level code splitting — bundles are split at import boundaries. Only load component code when the route is visited, reducing initial bundle size. At HSBC dashboard: admin module loaded lazily — saved 200KB from initial bundle.

Q: What are Error Boundaries and when do you use them?

A: Error Boundaries are class components implementing `componentDidCatch` / `getDerivedStateFromError`. They catch rendering errors in child tree and display fallback UI instead of crashing the app. Use around route-level components so one section's error doesn't crash the entire app. Libraries: `react-error-boundary` for functional component API.

Q: How do you optimize React application performance?

A: 1) `React.memo` for pure components, 2) `useMemo`/`useCallback` where profiling shows unnecessary re-renders, 3) Virtualization (`react-window`) for long lists (transaction history), 4) Code splitting with `React.lazy`, 5) Avoid inline object/function creation in JSX, 6) Use keys correctly in lists, 7) Debounce search inputs.

Q: Explain the Axios interceptor pattern used in production.

A: Request interceptor: inject JWT token header. Response interceptor: handle 401 (refresh token), 500 (show generic error), network errors (offline banner). Create axios instance with baseURL; attach interceptors once — avoids repetition across all API calls.

React Code Patterns

```
// Axios instance with interceptors
const api = axios.create({ baseURL: '/api' });
api.interceptors.request.use(cfg => {
  cfg.headers.Authorization = `Bearer ${getToken()}`;
  return cfg;
});
api.interceptors.response.use(res => res, err => {
  if(err.response?.status === 401) logout();
  return Promise.reject(err);
});

// Protected Route
const PrivateRoute = ({ children }) => {
  const { isAuthenticated } = useAuth();
  return isAuthenticated ? children : ;
};
```

SECTION 6 | DATABASE & SQL MASTER SECTION

Indexes & Query Optimization

Q: How does a B-Tree index work and when should you NOT use one?

A: B-Tree index maintains sorted keys with pointers to rows — $O(\log n)$ lookup. Avoid indexes on: low-cardinality columns (boolean, status with few values), columns rarely used in WHERE, tables with very frequent INSERT/UPDATE (index maintenance overhead). Composite index: (account_id, txn_date) benefits WHERE account_id=? AND txn_date>? but not WHERE txn_date>? alone.

Q: Explain EXPLAIN / EXPLAIN ANALYZE in MySQL.

A: EXPLAIN shows query execution plan: type (ALL=full scan, ref=index lookup, eq_ref=primary key), possible_keys, key used, rows estimated. At HSBC: EXPLAIN revealed 'type: ALL' on transaction_date — adding index reduced response from 2.3s to 45ms.

SQL Joins

Q: What is the difference between INNER, LEFT, RIGHT, and FULL OUTER JOIN?

A: INNER: only matching rows from both tables. LEFT: all left rows + matching right (NULLs where no match). RIGHT: all right rows + matching left. FULL OUTER: all rows from both (MySQL uses UNION of LEFT and RIGHT). Self JOIN: join table to itself — e.g., find employees managed by same manager.

Transactions & ACID

Q: Explain ACID properties with a banking example.

A: Atomicity: transfer debit+credit both succeed or both fail (no partial). Consistency: balance never goes negative (constraint enforced). Isolation: concurrent transfers don't see each other's intermediate state (read committed/serializable). Durability: committed transaction survives crash (WAL/redo log). Spring @Transactional manages ACID at application level, delegating to DB.

Q: What is the difference between @Transactional propagation types?

A: REQUIRED (default): join existing TX or create new. REQUIRES_NEW: always new TX, suspends existing — use for audit logging (must persist even if main TX fails). NESTED: savepoint within existing TX. MANDATORY: must have existing TX. NEVER: must NOT have TX.

Window Functions & Advanced SQL

Q: Write a query to find the top 3 transactions per account.

A: `SELECT * FROM (SELECT *, ROW_NUMBER() OVER(PARTITION BY account_id ORDER BY amount DESC) rn FROM transactions) t WHERE rn <= 3;` Window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER) are frequently tested in product company SQL rounds.

Q: How do you implement pagination efficiently in MySQL?

A: Avoid LIMIT 10000, 20 (scans 10020 rows). Use keyset pagination: WHERE id > :lastId ORDER BY id LIMIT 20. For complex sorts, use a cursor-based approach. At HSBC, transaction history pagination used created_at + id composite cursor.

Important SQL Queries to Practice

■ 2nd highest salary

```
SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);
```

■ Duplicate rows

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

■ Running total

```
SELECT id, amount, SUM(amount) OVER(ORDER BY created_at) AS running_total FROM transactions;
```

■ Delete duplicates

```
DELETE FROM users WHERE id NOT IN (SELECT MIN(id) FROM users GROUP BY email);
```

■ Transactions in last 30 days per account

```
SELECT account_id, COUNT(*) txn_count, SUM(amount) total  
FROM transactions  
WHERE created_at >= NOW() - INTERVAL 30 DAY  
GROUP BY account_id ORDER BY total DESC;
```

SECTION 7 | MICROSERVICES & SYSTEM DESIGN

Monolith vs Microservices Trade-offs

Aspect	Monolith	Microservices
Deployment	Single unit	Independent per service
Scalability	Scale entire app	Scale individual services
Fault isolation	One failure can crash all	Failures are contained
Data management	Single shared DB	DB per service
Complexity	Lower operational complexity	Higher (service mesh, distributed tracing)
Best for	Small teams, early stage	Large teams, complex domains

Key System Design Concepts

Q: How would you design a scalable notification system for a bank?

A: Components: Kafka topics (email-notifications, sms-notifications, push-notifications), Notification Producer (transaction service publishes events), Notification Consumers (separate microservices per channel with retry logic), Template Service (renders message from template + event data), DLQ (Dead Letter Queue for failed notifications after N retries). This decouples transaction processing from notification delivery — transaction service doesn't wait for email to send.

Q: Explain the CAP theorem and where your HSBC system sits.

A: CAP: distributed system can guarantee at most 2 of: Consistency (all nodes see same data), Availability (every request gets response), Partition Tolerance (works despite network splits). Banking systems typically choose CP (Consistency + Partition Tolerance): we accept temporary unavailability over showing incorrect account balance. MySQL with synchronous replication = CP. Cassandra = AP.

Q: How do you handle distributed transactions across microservices?

A: Option 1: SAGA pattern — choreography (events trigger next step) or orchestration (central coordinator). Option 2: Two-Phase Commit (2PC) — strong consistency but slow, single point of failure. At HSBC scale: SAGA with compensating transactions. Fund transfer: debit account A (step 1) -> credit account B (step 2); if step 2 fails, compensate by crediting A back. Status tracked in a saga state store.

Q: How do you implement API rate limiting at the gateway level?

A: Spring Cloud Gateway + Redis: RequestRateLimiter filter using token bucket algorithm. Configure rate per IP, per user, per API key. Return 429 Too Many Requests with Retry-After header. For HSBC public APIs: 100 req/min per client, 1000 req/min per internal service.

Architecture: HSBC Core Banking Microservices (Typical Layout)

- API Gateway (Spring Cloud Gateway) — single entry point, auth filter, rate limiting, routing

- Identity Service — user auth, JWT issuance, OAuth2
- Account Service — account management, balance queries (Redis cached)
- Transaction Service — debit/credit processing, publishes to Kafka
- Notification Service — Kafka consumer, email/SMS/push delivery
- Fraud Detection Service — called via Feign from Transaction Service (Circuit Breaker)
- Reporting Service — read-side, consumes Kafka events, aggregates to reporting DB
- Config Server (Spring Cloud Config) — centralized config from Git
- Service Registry (Eureka) — service discovery
- Distributed Tracing (Zipkin/Micrometer) — trace requests across services

SECTION 8 | PROJECT DEEP DIVE

Project 1: HSBC Core Banking Platform — Capgemini

Domain: Core Banking | Team: Agile Scrum | Duration: Dec 2022 – Present

Q: Walk me through your role and contribution at HSBC.

A: I was a Senior Software Engineer in an Agile team building and maintaining 12+ Spring Boot microservices for HSBC's core banking. My primary contributions: (1) Implemented JWT/RBAC security across services, (2) Introduced Redis caching that reduced repeated DB calls on account summary pages, (3) Built React.js dashboard components with Axios API integration, (4) Resolved N+1 queries via entity graphs, (5) Maintained 85%+ test coverage, (6) Participated in production incident resolution using Splunk log analysis.

Q: How did you implement Redis caching and what was the business impact?

A: Account summary page was called ~200 times/min per peak hour. Each call hit MySQL. I introduced @Cacheable with 5-minute TTL on account summary DTO. Result: 70% of requests served from Redis, MySQL load reduced significantly, response time improved from ~250ms to ~20ms for cached responses. Cache invalidation: @CacheEvict on any account update operation.

Q: Describe a production incident you resolved.

A: Alert: Transaction service latency spiking to 8 seconds (SLA: 500ms). Used Splunk to identify slow queries — saw 'SELECT * FROM transactions WHERE account_id=? took 7.8s'. EXPLAIN showed full table scan — account_id column lacked index on transactions table. Added index in a no-downtime migration (ALTER TABLE ... ALGORITHM=INPLACE). Latency dropped to 45ms. Root cause: index dropped during schema migration 2 sprints earlier.

Q: How did you handle the Jenkins CI pipeline for your microservices?

A: Pipeline stages: Checkout -> Maven Build -> Unit Tests (JUnit 5/Mockito) -> Code Coverage Check (Jacoco >85%) -> Static Analysis (SonarQube) -> Docker Build & Push to registry -> Helm Deploy to staging K8s -> Integration Tests -> Manual approval -> Production rolling deployment. Used feature branches with PR merge triggering full pipeline.

Q: What challenges did you face and how did you overcome them?

A: Challenge 1: Microservice communication failures causing cascade failures. Solution: Implemented Resilience4j Circuit Breaker with fallback responses — isolated failures. Challenge 2: JWT token expiry causing mid-session logouts. Solution: Implemented refresh token mechanism with sliding window expiry. Challenge 3: React frontend state inconsistency after API errors. Solution: Centralized error handling via Axios interceptors + Redux error slice.

Project 2: Job Portal Application (Personal)

Q: Explain the architecture of your Job Portal.

A: Full-stack: React.js (SPA) -> Spring Boot REST APIs -> MySQL + JWT Auth + Docker. Modules: Job Seeker (browse jobs, apply, track applications), Recruiter (post jobs, review applicants), Admin (manage users). Role-based routing in React. Spring Boot: separate controllers for each role, method security with @PreAuthorize.

Q: What improvements would you make to productionize the Job Portal?

A: 1) Add Kafka for async notifications (application status updates), 2) Implement Elasticsearch for job search (full-text, filters), 3) Add Redis for session management and rate limiting, 4) AWS S3 for resume file uploads, 5) Spring Cloud Config for environment-specific configs, 6) Add monitoring with Spring Actuator + Prometheus + Grafana, 7) TestContainers for integration testing with real MySQL.

SECTION 9 | MUST-KNOW INTERVIEW QUESTIONS

Core Java Top Questions

1. What is the contract between equals() and hashCode()?	If <code>a.equals(b)</code> then <code>a.hashCode()==b.hashCode()</code> . Converse not required. Violation: objects added to <code>HashMap</code> may not be found if <code>hashCode</code> inconsistent with <code>equals</code> .
2. What is String interning and when does it happen?	String literals are interned (stored in String Pool in Metaspace). <code>'hello'=='hello'</code> is true. <code>new String('hello')</code> creates a heap object. Use <code>intern()</code> to add to pool. Java 8 moved pool from PermGen to Metaspace/heap.
3. Explain try-with-resources.	Implements <code>AutoCloseable</code> — resource closed automatically after try block. Suppressed exceptions: if both try and <code>close()</code> throw, <code>close()</code> exception is suppressed; access via <code>getSuppressed()</code> .
4. What is a WeakReference and when would you use it?	<code>WeakReference</code> allows GC to collect the referenced object when no strong references exist. Use in memory-sensitive caches (<code>WeakHashMap</code>) where entries should be freed under memory pressure.
5. Difference between Comparable and Comparator?	<code>Comparable</code> : natural ordering, implemented by the class itself (<code>compareTo</code>). One ordering. <code>Comparator</code> : external comparison — multiple orderings, passed to <code>sort/TreeSet</code> . Java 8: <code>Comparator.comparing(User::getName)</code> .
6. What is covariant return type?	Overriding method can return a subtype of the parent method's return type. Enables fluent builder patterns and more specific return types without casting.
7. What is the Executor Framework?	Separates task submission from execution. <code>ThreadPoolExecutor</code> manages pool of threads. Fixed, Cached, Scheduled, Single-thread pools via Executors factory. Use <code>ExecutorService.submit()</code> for Future results; <code>shutdown()</code> gracefully.
8. Explain Java Memory Model (JMM).	Defines how threads interact via memory. Key guarantees: volatile writes visible to subsequent volatile reads; synchronized releases visibility guarantee to acquiring thread. Happens-before relationship ensures no data races.

9. What is a ThreadLocal and when is it dangerous?	ThreadLocal stores per-thread values. Used for request context (request ID, user session) in Spring via RequestContextHolder. Danger: thread pool reuse — ThreadLocal values persist across requests. Always call remove() in finally block.
10. What is sealed class in Java 17?	Restricts which classes can extend/implement it. permits clause lists allowed subclasses. Enables exhaustive pattern matching in switch expressions. Example: sealed interface Shape permits Circle, Rectangle, Triangle.

Spring Boot Top Questions

1. What is the difference between @RestController and @Controller?	@RestController = @Controller + @ResponseBody on every method. Returns data (JSON/XML) directly. @Controller returns view names for template rendering.
2. How does @Transactional work under the hood?	Spring creates a proxy (JDK dynamic proxy or CGLIB). Proxy intercepts method call, begins TX, calls target method, commits or rolls back. Important: self-invocation bypasses proxy — @Transactional won't work on methods called within same class.
3. What is Spring Boot Actuator?	Provides production-ready endpoints: /health, /metrics, /info, /env, /beans, /httptrace. Integrates with Micrometer for Prometheus/Grafana. At HSBC, /health used by K8s liveness/readiness probes.
4. How do you write integration tests in Spring Boot?	@SpringBootTest + @AutoConfigureMockMvc for full context. Use @MockBean for external deps. TestContainers for real DB/Kafka. MockMvc for HTTP layer testing without running server.
5. What is lazy initialization in Spring Boot?	spring.main.lazy-initialization=true: beans created only when first accessed, not at startup. Faster startup time (useful for Lambda/serverless). Risk: runtime errors if misconfigured bean not caught at startup.
6. Explain Spring Cloud Config Server.	Centralized configuration from Git/filesystem. Clients fetch config on startup via spring.config.import. Supports encryption of sensitive properties. @RefreshScope on beans allows config refresh at runtime via /actuator/refresh endpoint.
7. What is the purpose of @EnableCaching?	Enables Spring's annotation-driven cache management. Works with any JSR-107 compatible provider (Redis, Caffeine, Ehcache). @Cacheable, @CacheEvict, @CachePut drive cache behavior.

8. How do you validate request bodies in Spring Boot?	Add @Valid or @Validated on method param. Use JSR-380 annotations on DTO: @NotNull, @Size, @Email, @Pattern. @ControllerAdvice catches MethodArgumentNotValidException and returns structured validation errors.
--	--

Microservices Top Questions

1. What is service discovery and why is it needed?	Dynamic environments (K8s, cloud) mean service instances come and go with changing IPs. Eureka: services register on startup, deregister on shutdown. Clients query Eureka to find healthy instances. K8s has built-in DNS-based discovery (Services), so Eureka less common in pure K8s deployments.
2. Explain the Saga pattern.	Manages distributed transactions without 2PC. Choreography: each service publishes events triggering the next. Orchestration: central saga orchestrator tells each service what to do and handles failures. Each step has a compensating transaction for rollback.
3. What is the Outbox pattern?	Ensures atomicity between DB write and event publish. Write event to an outbox table in same TX as business data. Outbox poller (or Debezium CDC) reads and publishes events. Prevents lost events if broker is down during publish.
4. How do you implement health checks for microservices in K8s?	Liveness probe: /actuator/health/liveness — K8s restarts container if it fails. Readiness probe: /actuator/health/readiness — K8s stops sending traffic if not ready. Startup probe: for slow-starting apps, avoids premature liveness failures.
5. What is distributed tracing and how do you implement it?	Assigns a trace ID to request at entry point (gateway); propagated via headers across services. Spans represent operations within each service. Micrometer Tracing (Spring Boot 3) + Zipkin or Jaeger collector. At HSBC: Splunk also used for log correlation via MDC-injected trace IDs.

SECTION 10 | SCENARIO-BASED QUESTIONS

SCENARIO: Your microservice memory usage keeps growing over 48 hours until OOM.

Step-by-Step Resolution:

- Step 1: Check Splunk/metrics: is heap growing continuously (memory leak) or just high stable usage?
- Step 2: If growing: capture heap dump (jmap or /actuator/heapdump).
- Step 3: Analyze in Eclipse MAT: look for retained heap — likely a static collection, ThreadLocal, or event listener not removed.
- Step 4: Common causes in Spring: @Autowired static fields, Hibernate session cache not cleared, Kafka consumer accumulating records.
- Step 5: Fix & deploy with memory leak resolved. Add JVM heap metrics alert for early warning.

SCENARIO: API response time degrades from 100ms to 3 seconds after a new deployment.

Step-by-Step Resolution:

- Step 1: Check Splunk for slow query logs immediately post-deploy.
- Step 2: Compare deployment diff — was a new JPQL query added? Was an index affected by a migration?
- Step 3: Use EXPLAIN on identified slow query.
- Step 4: Check if N+1 problem introduced — Hibernate show_sql=true in staging.
- Step 5: If not DB: check external service call — did Feign client timeout increase? Is circuit breaker opening?
- Step 6: Rollback if business-critical; hotfix and re-deploy with index/query fix.

SCENARIO: React dashboard shows stale data — user updates account, refreshes, sees old balance.

Step-by-Step Resolution:

- Step 1: Check if Redis cache TTL is too long — balance showing cached data.
- Step 2: Verify @CacheEvict is called on balance update API endpoint.
- Step 3: If cache is correct: check React state — is component re-fetching after update?
- Step 4: Check Axios call — is response cached by browser? Add Cache-Control: no-cache headers.
- Step 5: Long-term: consider WebSocket/SSE for real-time balance updates in a banking UI.

SCENARIO: Kafka consumer is lagging — messages accumulating in topic.

Step-by-Step Resolution:

- Step 1: Check consumer group lag via kafka-consumer-groups.sh or Kafka UI.
- Step 2: Is the consumer slow (heavy processing per message) or dead (no consumers running)?

- Step 3: If slow: increase parallelism — add more partitions and consumer instances (up to partition count).
- Step 4: Check for blocking calls in consumer — DB calls, external HTTP — make async if possible.
- Step 5: If dead: check consumer logs for exceptions, uncaught errors causing consumer to exit.
- Step 6: Add dead letter topic for messages that fail after N retries.

SCENARIO: JWT tokens are being accepted after user logout.

Step-by-Step Resolution:

- Step 1: JWT is stateless — server can't invalidate issued tokens.
- Step 2: Solution 1: Short TTL (15 min) + refresh token rotation — logout invalidates refresh token in Redis.
- Step 3: Solution 2: Token blacklist in Redis — on logout, store jti (JWT ID) in Redis with TTL = token expiry.
- Step 4: On each request: check jti against Redis blacklist before accepting token.
- Step 5: Trade-off: blacklist adds Redis lookup per request — acceptable for banking security requirements.

SECTION 11 | CODING ROUND PREPARATION

DSA Priority Roadmap for 3.5Y Java Developer

At your experience level, product companies expect medium-difficulty DSA proficiency. Focus on patterns over memorization. Daily 30-45 min practice for 60 days is sufficient for most interview filters.

Topic	Priority	Key Patterns	Target Problems
Arrays & Strings	HIGH	Two pointers, sliding window	Max subarray, anagram, longest substring
HashMap/HashSet	HIGH	Frequency count, lookup	Two sum, group anagrams, subarray sum
Stack & Queue	HIGH	Monotonic stack, BFS	Valid parentheses, next greater element, LRU Cache
Linked List	MEDIUM	Fast/slow pointer, reversal	Cycle detect, reverse LL, merge sorted
Binary Search	HIGH	Search in rotated, boundary	Rotated array search, kth element
Trees (BFS/DFS)	MEDIUM	Level order, inorder, path	LCA, path sum, zigzag traversal
Recursion & DP	MEDIUM	Memoization, tabulation	Fibonacci, coin change, house robber
Sorting	LOW	Merge sort, quick sort	Know time complexities; rarely coded
Graphs	LOW	BFS, DFS, Union-Find	Islands, connected components

Important Coding Problems with Solutions

■ Two Sum (HashMap — $O(n)$)

```
Map seen = new HashMap<>();
for(int i=0; i < nums.length; i++){
    int comp = target - nums[i];
    if(seen.containsKey(comp)) return new int[]{seen.get(comp), i};
    seen.put(nums[i], i);
}
return new int[]{};
```

Time: $O(n)$ | Space: $O(n)$

■ Longest Substring Without Repeating (Sliding Window)

```
Set set = new HashSet<>();
int l=0, max=0;
for(int r=0; r < s.length(); r++){
    while(set.contains(s.charAt(r))) set.remove(s.charAt(l++));
    set.add(s.charAt(r)); max=Math.max(max, r-l+1);
}
return max;
```

Time: $O(n)$ | Space: $O(\min(n, \text{charset}))$

■ Valid Parentheses (Stack)

```
Deque stack = new ArrayDeque<>();
Map map = Map.of(')', '(', ']', '[', '}', '{');
for(char c: s.toCharArray()){
    if(map.containsKey(c)){
        if(stack.isEmpty() || stack.peek() != map.get(c)) return false;
        stack.pop();
    } else stack.push(c);
}
return stack.isEmpty();
```

Time: $O(n)$ | Space: $O(n)$

■ Detect Cycle in Linked List (Floyd's)

```
ListNode slow=head, fast=head;
while(fast!=null && fast.next!=null){
    slow=slow.next; fast=fast.next.next;
    if(slow==fast) return true;
}
return false;
```

Time: $O(n)$ | Space: $O(1)$

■ Binary Search

```
int l=0, r=nums.length-1;
while(l<=r){
    int mid=l+(r-l)/2;
    if(nums[mid]==target) return mid;
    if(nums[mid] < target) l=mid+1;
    else r=mid-1;
}
return -1;
```

Time: $O(\log n)$ | Space: $O(1)$

SECTION 12 | HR & MANAGERIAL ROUND PREPARATION

Q: Tell me about yourself.

A: I'm Viraj Pawar, a Java Full Stack Developer with 3.5 years of experience at Capgemini, working on HSBC's core banking platform. I specialize in Java 17/21, Spring Boot microservices, and React.js frontends. In my current role, I've contributed to 12+ production microservices handling account management, transactions, and user authentication at HSBC scale. I hold Oracle's Java SE 21 certification and AWS Cloud Practitioner, reflecting my commitment to staying current. I'm now looking for a role where I can take greater ownership of architecture decisions and grow into a senior engineering position.

Q: Why are you leaving Capgemini?

A: I've had a great 3.5 years at Capgemini and learned a tremendous amount on HSBC's banking platform. I'm now looking for a role with more direct product ownership, architectural exposure, and opportunities to work on greenfield systems, which is harder to get in a services-company environment where client decisions drive the roadmap.

Q: Describe a conflict with a team member and how you resolved it.

A: During a sprint, a colleague and I disagreed on the approach for the caching layer — they preferred EhCache (simpler), I recommended Redis (distributed, supports multi-pod). I prepared a concise comparison with pros/cons for our use case and presented to the tech lead. We went with Redis. Key lesson: let data and use-case requirements drive decisions, not personal preference.

Q: Where do you see yourself in 3 years?

A: I want to grow into a Senior Engineer or Tech Lead role where I'm driving architectural decisions, mentoring junior developers, and contributing to technical roadmaps. I aim to deepen my expertise in distributed systems design and cloud-native architecture, while continuing to deliver business value through engineering excellence.

Q: What is your expected CTC?

A: Based on my experience, skills, and market research for Java Full Stack roles at this level, I'm targeting between X and Y LPA. I'm flexible based on the full compensation structure, growth opportunities, and work environment. What is the budgeted range for this role?

Q: What is your biggest weakness?

A: I sometimes spend more time than necessary trying to find the optimal solution before starting, when an iterative approach would deliver value faster. I've been actively addressing this by using time-boxing: I give myself 30 minutes to design, then start coding and refine iteratively, which has improved my delivery speed significantly.

Q: How do you handle production pressure?

A: I stay methodical. When an incident fires: (1) Acknowledge the alert immediately, (2) Assess impact, (3) Check Splunk logs and identify root cause hypothesis, (4) Apply targeted fix or rollback if needed, (5) Communicate status to stakeholders, (6) Post-mortem to prevent recurrence. At HSBC, I handled a transaction service outage by identifying a missing DB index within 20 minutes and deploying a hotfix.

SECTION 13 | DAILY STUDY PLAN — 30 & 60 DAY ROADMAP

30-Day Intensive Roadmap

Week	Focus Area	Daily Tasks	Daily Time
Week 1 (Days 1-7)	Java Core + Collections + OOPs	Revise Streams, Lambdas, Optional, Collections internals, Generics. Practice 2 Java interview Qs/day. 2 LeetCode Easy (arrays, strings).	2.5 hrs
Week 2 (Days 8-14)	Spring Boot + Security + JPA	Deep dive: auto-config, bean lifecycle, Spring Security filter chain, JWT. Revise JPA fetch strategies, transaction management. 2 LeetCode Medium.	2.5 hrs
Week 3 (Days 15-21)	Microservices + Kafka + System Design	Study SAGA, CQRS, outbox pattern. Build basic Kafka producer-consumer locally. Read 1 system design case study. 2 LeetCode Medium.	3 hrs
Week 4 (Days 22-30)	React.js + SQL + Mock Interviews	Redux Toolkit, React Query, performance optimization. SQL window functions, execution plans. 2 full mock interviews (DSA + HR).	3 hrs

60-Day Extended Roadmap

Days 1–30: Follow 30-day plan above. Days 31–60:

- **Days 31–37:** AWS deep dive (EC2, RDS, S3, ECS/EKS). Complete one AWS hands-on project.
- **Days 38–44:** GraphQL basics, TestContainers, advanced Spring Boot testing.
- **Days 45–51:** System Design: design Twitter, URL shortener, payment system. Draw diagrams.
- **Days 52–56:** LeetCode: 3 mediums/day focused on trees, DP, binary search.
- **Days 57–60:** Full mock interview sets. Review weak areas. Final revision of cheat sheets.

DAILY RHYTHM: Morning (30 min): 1 LeetCode problem. Evening (90 min): Study guide + code.
Weekend: 1 mock interview + 1 system design exercise. Consistency > intensity.

SECTION 14 | RAPID REVISION NOTES & CHEAT SHEETS

Java One-Liners

Term/Concept	Quick Definition
HashMap	Hash table; O(1) avg; not thread-safe; allows null key/values
ConcurrentHashMap	Thread-safe; no null keys; uses CAS + bucket locking
volatile	Visibility guarantee; no atomicity
synchronized	Mutual exclusion + visibility; intrinsic lock
CompletableFuture	Async task chaining; non-blocking; supports compose, combine, exceptionally
Stream.map vs flatMap	map: 1-to-1; flatMap: 1-to-many + flatten
@Transactional self-call	DOESN'T work — proxy bypassed; move to separate bean
N+1 fix	@EntityGraph or JOIN FETCH in JPQL or @BatchSize
Circuit Breaker states	CLOSED (normal) -> OPEN (fail fast) -> HALF_OPEN (test recovery)
JWT structure	header.payload.signature — stateless; cannot revoke without blacklist
Kafka offset	Consumer position; commit after processing; auto vs manual commit
Spring profiles	@Profile('prod') or spring.profiles.active=prod; per-env config
@Cacheable	Cache method result; key by params; TTL via cache config
React useEffect []	Runs once on mount — equivalent to componentDidMount
React.memo	Prevents re-render if props unchanged — shallow comparison
SQL EXPLAIN type ALL	Full table scan — add index on WHERE column
ACID: I = Isolation	READ COMMITTED prevents dirty reads; SERIALIZABLE prevents phantoms
G1GC	Default Java 9+; region-based; targets low pause (-XX:MaxGCPauseMillis=200)

HTTP Status Codes Cheat Sheet

Status Code	Meaning
-------------	---------

200 OK	Request succeeded
201 Created	Resource created
204 No Content	Success, no body
400 Bad Request	Client error, invalid input
401 Unauthorized	Not authenticated
403 Forbidden	Authenticated but not authorized
404 Not Found	Resource doesn't exist
409 Conflict	Duplicate resource
422 Unprocessable	Validation failed
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Generic server error
503 Service Unavailable	Circuit breaker open / overloaded

Design Patterns Quick Reference

Pattern	Type	One-Line Use
Singleton	Creational	One instance — Spring @Bean default scope
Factory / Factory Method	Creational	Delegate object creation — payment type factory
Builder	Creational	Complex object with many optional fields — Lombok @Builder
Strategy	Behavioral	Interchangeable algorithm — payment processor, sorting
Observer	Behavioral	Event notification — Spring ApplicationEvent
Template Method	Behavioral	Algorithm skeleton — JdbcTemplate
Proxy	Structural	Access control / logging — Spring AOP, @Transactional
Decorator	Structural	Add behavior dynamically — Java I/O streams
Circuit Breaker	Resilience	Fail fast when downstream fails — Resilience4j
Outbox	Data	Atomic DB write + event publish — prevent lost events
SAGA	Distributed TX	Long-running transactions without 2PC

FINAL CONFIDENCE TIP: Viraj, your HSBC banking experience is a significant differentiator. In every interview, anchor your answers to real production scenarios — caching 70% of DB reads, resolving a transaction latency incident, implementing JWT RBAC. Interviewers at senior levels value real-world judgment over textbook answers. You have it. Own it.

Good luck, Viraj! — Your Personalized Interview Bible